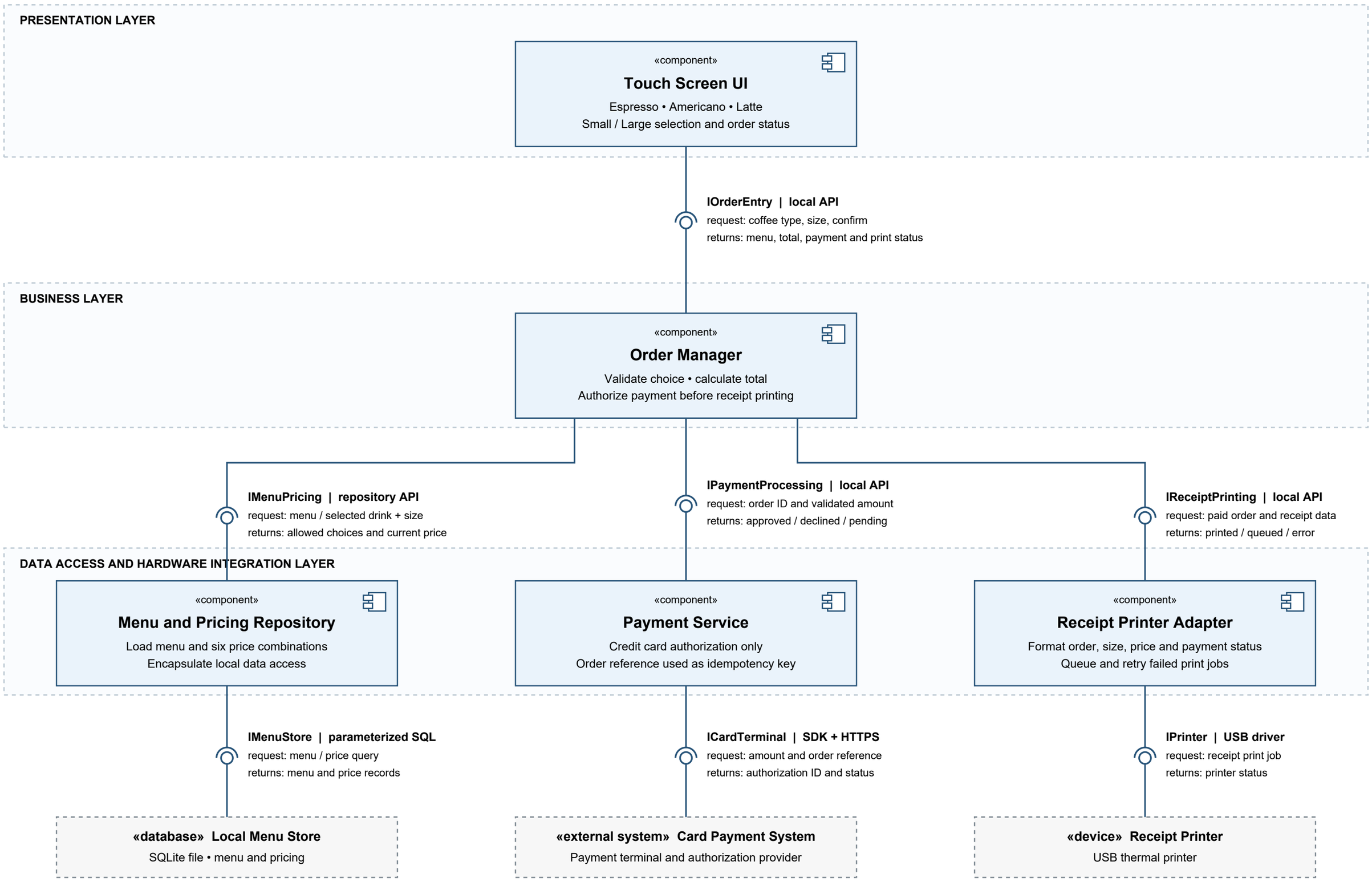
LAB 3 | Sampreeth Jaladi | PES1UG24AM246

Self-Service Coffee Kiosk System

UML Component Diagram • Layered Architecture



Legend: open ball = provided interface • socket = required interface • solid connection = assembly • requests go toward the provider; responses return

Lab 3 Architecture Justification

Self-Service Coffee Kiosk System

Sampreeth Jaladi | SRN PES1UG24AM246

Architecture Selection

We chose Layered Architecture for the Self-Service Coffee Kiosk System. The design separates the touch interface, ordering rules, and data and hardware integration. It supports Espresso, Americano and Latte in Small or Large sizes, credit-card payment, and printed receipts.

Reason 1 Changes remain local to each responsibility

The Touch Screen UI displays choices and sends selections to the Order Manager. The Menu and Pricing Repository supplies the current price for each drink and size, while the Payment Service and Receipt Printer Adapter isolate their device interfaces. A change to a price, payment integration or printer driver can therefore be made in its own component while keeping the other interface contracts stable. This makes the kiosk easier to test and maintain.

Reason 2 The architecture matches the scale of one kiosk

This application has a small, fixed menu and a single local ordering workflow. One application with well-defined layers is simpler to deploy and diagnose than independently deployed microservices, which would introduce network calls and distributed failure handling without a clear scaling need. A remote client-server design could centralize control across many kiosks, but would make menu access and ordering more dependent on connectivity. Local menu storage suits this kiosk; card authorization still requires the payment provider.

Security Advantage

The Order Manager validates the selected drink and size and calculates the amount from stored prices rather than trusting a total supplied by the UI. The Payment Service alone invokes the payment terminal and provider. Card entry and card credentials remain in the payment system; the kiosk receives only an authorization reference and status. These boundaries reduce sensitive-data exposure, provided they are enforced in the implementation. Logs and receipts exclude full card numbers and security codes.

Performance Benefit

Internal components use in-process API calls, and the repository reads the small menu from local storage. This avoids network round trips for browsing and price lookup. Payment and printing run asynchronously so the touch screen stays responsive. The Order Manager prints only after confirmed approval, uses the order ID to prevent duplicate payment requests, and lets the printer adapter retry a failed print without charging again.

Component Interaction

The UI requires IOrderEntry from the Order Manager. The Order Manager requires IMenuPricing, IPaymentProcessing and IReceiptPrinting from their respective components. The repository, payment service and printer adapter then use the external storage and hardware interfaces shown in the diagram. Payment decline or unresolved authorization prevents a paid receipt from being issued.